

PureScript Alternative Backends: Cross-Backend Comparison

Status: active **Category:** research **Created:** 2026-02-24 **Author:** Claude (synthesis of Racket port report, purerl source, purepy docs, .NET feasibility study)

1. Executive Summary

PureScript compiles to CoreFn, an untyped functional intermediate representation, which alternative backends then translate to their target language. Five backends are covered here:

Backend	Target	Status	Maintained By
purs (reference)	JavaScript	Production	PureScript core team
purerl	Erlang/BEAM	Production	id3as (Rob Sheridan et al.)
purs-backend-erl	Erlang/BEAM	Production	id3as (rewrite of purerl)
purescm	Chez Scheme / Racket	Experimental	Nathan Faubion (aristanetworks)
purepy	Python	Working (90+ tests)	Claude-authored, afc maintained
.NET (hypothetical)	CLR / C#	Researched, not built	—

Note on purerl vs purs-backend-erl: There are two Erlang backends. The original **purerl** ([purerl/purerl](#)) is Haskell-based and reads CoreFn directly. The newer **purs-backend-erl** ([id3as/purescript-backend-erl](#)) is a PureScript-based rewrite that uses [purescript-backend-optimizer](#), achieving ~30-40% performance improvements. Both are maintained by id3as and are FFI-compatible. The local codebase at [purescript-backends/purerl/](#) is the original Haskell version.

Additionally, **purescript-lua** ([pslua](#)) exists as an experimental Lua backend and **purekt** shares architecture with purescm via [purescript-backend-optimizer](#).

Who this is for: Backend implementors, library authors writing portable PureScript, and anyone evaluating PureScript for a non-JS target.

Key insight: The dominant factor in backend viability is not the compilation strategy — it's the **FFI rewrite cost**. Every backend must reimplement ~50 core FFI modules from scratch. This is the single largest effort and the primary reason most alternative backends stall.

2. Architecture Comparison

All alternative backends share a common pipeline structure:

PureScript Source

→ purs compile (--codegen corefn)

→ CoreFn JSON (untyped functional IR)

→ [purescript-backend-optimizer] ← optional but recommended

→ Backend-specific code generator

→ Target language source / bytecode

Dimension	JS (reference)	Purerl (original)	Purs-backend-erl	Purescm/Purekt	Purepy	.NET (proposed)
-----------	----------------	-------------------	------------------	----------------	--------	-----------------

Dimension	JS (reference)	Purerl (original)	Purs-backend-erl	Purescm/Purekt	Purepy	.NET (proposed)
Implementation language	Haskell (in purs)	Haskell	PureScript	PureScript	Haskell + PureScript	Haskell (proposed)
Uses backend-optimizer	No (own passes)	No (own Haskell passes)	Yes	Yes (originated here)	Yes	Would use it
IR consumed	CoreFn	CoreFn	Optimized CoreFn	Optimized CoreFn	Optimized CoreFn	CoreFn
Output format	ES modules / CJS	.erl source in output/	.erl source in output-erl/	.rkt / .ss source	.py source	C# source or CIL
Invocation	Built into purs	backend.cmd in spago.yaml	Two-step: spago build then purs-backend-erl	CLI post-pass	CLI post-pass	—
Bundling	esbuild/webpack	rebar3/mix	rebar3/mix	raco	pip/setuptools	dotnet build
FFI language	JavaScript	Erlang	Erlang (compatible)	Scheme/Racket	Python	C#
Performance	Baseline	Baseline (Erlang)	~30-40% faster (uncurrying, inlining)	—	—	—

The Role of **purescript-backend-optimizer**

Created by Nathan Faubion at Arista Networks for purescm/purekt, this PureScript library transforms CoreFn into an optimized IR with:

- **Aggressive inlining** — subsumes the PureScript compiler's own inlining
- **Uncurrying** — tracks function arities across modules, generates direct calls
- **Pattern matching optimization** — eliminates redundant tests
- **Directive system** — backends can annotate functions (e.g., **tailRec never**)
- **Lighter data encoding** — plain objects with string/int tags

Purerl adopted it for its PureScript-based rewrite. Purepy uses it via its Haskell integration. The JS reference backend does not use it (it has its own optimization passes in the compiler).

3. Semantic Mapping: Data Representation

The core question for any backend: how do you represent PureScript's data types in the target language?

Data Representation Table

PureScript Type	JS (reference)	Purerl (Erlang)	Purescm (Scheme)	Purepy (Python)	.NET (proposed)
-----------------	----------------	-----------------	------------------	-----------------	-----------------

PureScript Type	JS (reference)	Purerl (Erlang)	Purescm (Scheme)	Purepy (Python)	.NET (proposed)
Int	JS number (53-bit)	<code>integer()</code> (arbitrary)	Fixnum/exact int	<code>int</code> (arbitrary)	<code>int</code> (32-bit)
Number	JS number (f64)	<code>float()</code>	Flonum	<code>float</code>	<code>double</code>
Boolean	<code>true/false</code>	<code>true/false</code> atoms	<code>#t/#f</code>	<code>True/False</code>	<code>bool</code>
String	JS string (UTF-16)	<code>binary()</code> (UTF-8)	Racket string (UCS-4)	<code>str</code> (Unicode)	<code>string</code> (UTF-16)
Char	JS string (length 1)	Integer (codepoint)	Racket char	<code>str</code> (length 1)	<code>char</code> (UTF-16)
Array a	JS Array	<code>array:array(T)</code>	Scheme vector	<code>list</code>	<code>T[]</code> or <code>ImmutableArray<T></code>
Record { ... }	JS Object	<code>#{atom => any()}</code> (map)	Scheme hash-table	<code>dict</code>	<code>Dictionary<string, object></code>
ADTs	<code>{tag, _1, _2, ...}</code>	<code>{tag, field1, field2}</code> (tuple)	<code>(vector tag f1 f2)</code>	<code>("Tag", f1, f2)</code> (tuple)	Abstract class + subclasses
Functions	JS function	<code>fun(X) -> ...</code> (closure)	Scheme lambda	<code>lambda x: ...</code>	<code>Func<T1, TResult></code>
Effect a	<code>() => a</code>	<code>fun() -> a</code>	<code>(lambda () ...)</code>	<code>lambda: a</code>	<code>Func<T></code>
Newtype	Transparent (erased)	Transparent (erased)	Transparent (erased)	Transparent (erased)	Transparent (erased)

Key Observations

- **Dynamic targets** (JS, Erlang, Scheme, Python, Lua) represent ADTs as tagged tuples/arrays/vectors. This is lightweight but untyped at runtime.
- **Static targets** (.NET) would benefit from class hierarchies (as F# demonstrates) for pattern matching and IDE support, at the cost of more allocation.
- **Records** are universally dictionaries/maps/objects — no backend has found a more efficient representation that preserves row polymorphism.
- **Newtypes** are universally erased — all backends handle this correctly.

4. Currying Strategy

Currying is the dominant performance concern for PureScript backends, especially on static-typed targets where closures are heap-allocated.

Currying Strategy Table

Dimension	JS	Purerl	Purescm	Purepy	.NET (proposed)
Default representation	Nested arrow functions	Nested <code>fun(X) -> ...</code>	Nested <code>(lambda (x) ...)</code>	Nested <code>lambda x: ...</code>	Nested <code>Func<></code> delegates

Dimension	JS	Purerl	Purescm	Purepy	.NET (proposed)
Uncurrying optimization	In-compiler passes	Backend-optimizer + arity overloads	Backend-optimizer	Backend-optimizer	Backend-optimizer (essential)
Partial application cost	Very low (closures cheap)	Low (BEAM closures lightweight)	Low (Scheme closures cheap)	Low (Python closures cheap)	Moderate-high (delegate alloc)
Saturated call optimization	Direct call	Generates arity-N overload	Direct call via optimizer	Via optimizer	Must uncurry aggressively
FFI auto-currying	N/A (JS is already curried)	Yes — arity-N exports auto-wrapped	N/A	Manual currying in FFI	Would need wrapper generation
Benchmark overhead	1x (baseline)	~1-2x (negligible)	~1-2x (negligible)	~18-22x (fib), ~3x (trees)	Unknown (estimated 3-5x)

Purerl's Arity Overload Strategy

Purerl generates **multiple overloads** for each function:

```
% Curried form (always generated)
add() -> fun(X) -> fun(Y) -> X + Y end end.

% Uncurried overload (generated when fully-applied call sites exist)
add(X, Y) -> (add())(X)(Y).
```

This means Erlang code calling PureScript can use the natural arity-2 form, while PureScript's own partial application still works through the curried form.

Purepy's Current Overhead

Python's overhead is disproportionately high (18-22x for fib) because:

- 1. Each curried call creates a new lambda object
- 2. Type class dictionaries add implicit extra arguments
- 3. Lazy thunks add indirection
- 4. No uncurrying optimization is applied at the Python level yet

The backend-optimizer handles cross-module uncurrying at the CoreFn level, but many call sites still use curried forms in the generated Python.

5. Tail Call Optimization

Tail call optimization is critical for PureScript, which generates deeply recursive code from `tailRec`, `foldl`, and other standard combinators.

Backend	TCO Approach	Mutual Recursion	Stack Depth Limit
JS	Trampoline (<code>tailRec</code>)	Trampoline only	~10,000 (V8)
Purerl	Native BEAM TCO	Yes (native)	Unlimited
Purescm	Native Scheme TCO	Yes (native)	Unlimited

Backend	TCO Approach	Mutual Recursion	Stack Depth Limit
Purepy	While-loop trampoline via optimizer directive	Trampoline only	~1,000 (CPython default)
.NET	.tail CIL prefix (if emitting IL) or trampoline	CIL: yes; C#: no	Unlimited (with .tail)

Python's Solution

Python's default recursion limit (~1000 frames) is the tightest constraint of any target. Purepy solves this via:

1. An optimizer directive `Control.Monad.Rec.Class.tailRec` never prevents inlining
2. The runtime provides a while-loop trampoline:

```
def tailRec(f):
    result = f(initial)
    while result[0] == 'Loop':
        result = f(result[1])
    return result[1]
```

3. Tested with 50,000+ element BST operations without stack overflow

.NET Considerations

If emitting CIL directly, the `.tail` prefix instruction provides native TCO. However, Roslyn (the C# compiler) never emits `.tail` — so a C#-emitting backend would need its own trampoline for mutual recursion. Self-recursion can use `while` loops.

6. Module System & Lazy Bindings

Module Naming

Backend	PureScript <code>Data.Maybe</code> becomes	Foreign module
JS	<code>Data.Maybe/index.js</code>	<code>Data.Maybe/foreign.js</code>
Purerl	<code>data_maybe@ps</code>	<code>data_maybe@foreign</code>
Purescm	<code>Data.Maybe.ss</code>	<code>Data.Maybe.foreign.ss</code>
Purepy	<code>data_maybe.py</code>	<code>data_maybe_foreign.py</code>

Forward References / Mutual Recursion at Module Level

PureScript's type class instances frequently form mutual reference cycles. Each backend handles this differently:

Backend	Strategy	Mechanism
JS	Variable hoisting	JS <code>var</code> declarations are hoisted; values assigned later
Purerl	Native	Erlang functions can reference functions defined later in the module
Purescm	Scheme <code>define</code>	Scheme's <code>define</code> naturally handles forward references
Purepy	<code>_runtime_lazy</code> thunks	Explicit lazy thunk machinery; lambdas reference <code>_lazy_X()</code> not <code>X</code>
.NET	Class statics	C# static fields with lazy initialization

Python has the most complex solution here because it executes module-level code sequentially with no hoisting. The `_runtime_lazy` pattern adds runtime overhead to every recursive binding group.

7. Pattern Matching

Backend	Code Generation Strategy	Native Pattern Matching
JS	Nested ternary expressions	No
Purerl	Erlang <code>case</code> expressions	Yes (native, with guards)
Purescm	Scheme <code>cond/match</code>	Yes (via Racket <code>match</code> or <code>cond</code>)
Purepy	Nested conditional expressions + walrus operator	Partial (Python 3.10 <code>match</code> is a statement, not expression)
.NET	<code>switch</code> expressions (C# 8+)	Yes (with pattern matching extensions)

Purepy's Approach

Pattern matching compiles to nested ternary expressions with lambdas for variable binding:

```
(lambda __v__:
  (((x := __v__[1]), body_using_x)[-1]
   if __v__[0] == "Just" else
   default_body)
)(scrutinee)
```

This is correct but generates dense, hard-to-debug code. A future optimization would use Python 3.10+ `match` statements for top-level case expressions (where the expression-vs-statement distinction doesn't matter).

8. Effect System & Async

Every backend faces the question: how to represent PureScript's `Effect` and `Aff` monads?

Effect Representation

All backends use the same approach: `Effect a` is a zero-argument function that returns `a`.

```
JS:      () => value
Erlang:  fun() -> Value end
Scheme:  (lambda () value)
Python:  lambda: value
.NET:    Func<T> or () => value
```

Async Monad

Backend	Async Approach	Platform Integration
JS	<code>Aff</code> (purescript-aff)	Node.js/browser event loop
Purerl	OTP processes + <code>gen_server</code>	Erlang's native concurrency
Purescm	Not yet implemented	Would use Racket threads or futures
Purepy	<code>Control.Monad.Asyncio</code>	Python's native <code>asyncio</code>
.NET	Would use <code>Task<T></code>	.NET's native <code>async/await</code>

Key lesson: Every backend that has gotten far enough to need async has built a **platform-native** async monad rather than porting Aff. Aff's semantics are deeply tied to JavaScript's event loop, and porting it faithfully requires reimplementing JavaScript's microtask queue — which is both unnatural and unnecessary when the target platform has its own async runtime.

9. FFI Deep Dive

The FFI is the **single most important factor** in backend viability. It determines:

1. How much work is needed to bootstrap (rewriting core library FFI)
2. How easily users can access platform libraries
3. Whether the backend can sustain itself as a community project

FFI Cost Table

Backend	Core FFI Files Needed	Current Status	Approach	Test Coverage
JS	~50 (reference)	Complete	JS modules alongside .purs	Comprehensive (compiler test suite)
Purerl	~50 (forked packages)	Complete	.erl files with auto-currying	Production use at id3as
Purescm	~50	Partial	.ss/.rkt files	Test suite (per Racket report)
Purepy	~50 (38 fully done)	Partial (~76%)	_foreign.py files, manual currying	90 tests in test_ffl.py
.NET	~50	Not started	Would be .cs files	—

The FFI Rewrite Cost Problem

For a new backend, the work breaks down roughly:

1. **Core compilation** (CoreFn → target): 2-4 weeks for a working prototype
2. **Core FFI rewrite** (Effect, Prelude, arrays, strings, etc.): 4-8 weeks
3. **Extended FFI** (JSON, regex, HTTP, testing): 4-8 weeks
4. **Ongoing maintenance:** Every PureScript compiler release may change CoreFn

The FFI rewrite is larger than the compiler itself. This is why the Racket port report's **dependency-wave** approach is valuable: implement FFI in topological order of package dependencies, testing as you go, rather than trying to implement everything at once.

Dependency-Wave Pattern (from Racket Report)

Wave 1: prelude, effect, console	(fundamental)
Wave 2: maybe, either, tuples	(data types)
Wave 3: arrays, strings, foldable	(collections)
Wave 4: refs, st, exceptions	(stateful effects)
Wave 5: aff/async, json, http	(ecosystem access)

Each wave's FFI is tested before the next begins. This catches semantic divergences early.

FFI Ergonomics Comparison

Dimension	JS	Purerl	Purepy
Auto-carrying	N/A (JS already curried)	Yes — exports auto-wrapped by arity	No — manual currying required
Type specs	None	.hrl files for Dialyzer	None (future: .pyi stubs)
FFI validation	Compiler checks file exists	Compiler checks exports	FFI canary module (compile-time)
Uncurried FFI	FnN/EffectFnN types	FnN/EffectFnN → direct arity-N	Not yet implemented

10. String Semantics

String handling is a recurring challenge for every alternative backend because JavaScript's strings are **UTF-16 encoded**, and PureScript's `Data.String.CodeUnits` module exposes this encoding directly.

String Semantics Table

Backend	String Type	Native Encoding	CodeUnit Behavior	Regex Support	Notable Challenge
JS	JS String	UTF-16	Correct by definition	Native RegExp	— (reference)
Purerl	<code>binary()</code>	UTF-8	Must emulate UTF-16 indexing	Erlang <code>re</code>	UTF-8 ↔ UTF-16 translation
Purescm	Racket string	UCS-4 (code points)	Must emulate UTF-16 indexing	Racket <code>regexp</code>	Code point ≠ code unit for non-BMP
Purepy	<code>str</code>	Unicode (code points)	Diverges for non-BMP	Python <code>re</code>	<code>len(s)</code> counts code points, not code units
.NET	<code>System.String</code>	UTF-16	Matches JS natively	<code>System.Text.RegularExpressions</code>	Free — .NET strings are UTF-16

Three Approaches to String Semantics

- Lucky match** (.NET): The target language happens to use UTF-16 internally. `Data.String.CodeUnits` operations work correctly with no effort.
- Faithful reimplementation** (Purerl, Purescm): Build UTF-16 encoding/decoding helpers so that `length`, `charAt`, `indexOf`, etc. all return UTF-16-based indices. Correct but adds complexity and potential performance overhead.
- Accept divergence** (Purepy, current): Use native string operations directly. Correct for BMP characters (U+0000 to U+FFFF) but silently wrong for emoji, CJK Extension B, musical symbols, and other non-BMP characters that require surrogate pairs in UTF-16.

Impact of Approach 3

For Purepy, the divergence affects every function in `data_string_code_units_foreign.py`:

- `length("😊 ")` returns 1 in Python, 2 in JS (surrogate pair)
- `charAt(0)("😊 ")` returns "😊 " in Python, "\uD83D" in JS (high surrogate)
- `indexOf`, `take`, `drop`, `slice`, `splitAt` — all use code-point offsets instead of code-unit offsets

This is documented in the UTF-16 String Audit (`purescript-python-new/docs/UTF16-STRING-AUDIT.md`).

Recommendation for new backends: Investigate string semantics on day one. If your target's native encoding doesn't match UTF-16, decide early whether to build faithful UTF-16 helpers or document the divergence.

11. Performance Characteristics

Available Benchmarks

Only Purepy has published cross-comparable benchmarks:

Benchmark	JS (node)	Purerl	Purepy	Notes
<code>fib(30)</code> naive recursive	~120ms	N/A	~2700ms (22x)	Dominated by currying + dict passing
<code>tree sum (depth 15)</code>	~20ms	N/A	~61ms (3x)	Dominated by allocation
<code>apply inc 100x</code>	~0.01ms	N/A	~0.20ms (20x)	Pure currying overhead
<code>sumTo 100</code>	~0.01ms	N/A	~0.18ms (18x)	Currying + recursion overhead

Where Each Backend Excels

Backend	Strengths	Weaknesses
JS	Fastest for most workloads; V8 JIT optimizes closures	Single-threaded without workers
Purerl	Concurrent workloads; fault tolerance; hot code reloading	Raw compute speed lower than JIT targets
Purescm	Scheme's native TCO; macro system for DSLs	Smaller ecosystem; less tooling
Purepy	Access to Python ML/data science ecosystem	Highest overhead (18-22x for compute)
.NET	JIT + NativeAOT; best raw perf potential after JS	Delegate allocation for curried functions

Performance Improvement Paths

Backend	Primary optimization opportunity
JS	Already well-optimized
Purerl	Type-spec-guided optimization; memoization
Purescm	Backend-optimizer already aggressive
Purepy	Uncurrying (2-3x), Smart Rec detection, PyPy (5-50x)
.NET	Uncurrying essential; NativeAOT for AOT compilation

12. Ecosystem Access & Deployment

Backend	Platform Ecosystem	Deployment Targets	Unique Access
JS	npm (~2M packages)	Browser, Node, Deno, Bun, Cloudflare Workers	Web frontends, serverless

Backend	Platform Ecosystem	Deployment Targets	Unique Access
Purerl	Hex/OTP	Distributed systems, telecom, messaging	OTP supervision trees, hot code reload
Purescm	Racket packages / Guile	Scripting, DSLs, language tooling	Macro system, language-oriented programming
Purepy	PyPI (~500K packages)	ML/AI, data science, scripting, Jupyter	NumPy, Pandas, PyTorch, TensorFlow
.NET	NuGet (~400K packages)	Azure, Unity, desktop, mobile (MAUI), WASM	Excel (COM), enterprise, game engines

The Ecosystem Argument

Each backend's value proposition is primarily about **what the target platform gives you access to**:

- Purerl: You want OTP's concurrency model with PureScript's type safety
- Purepy: You want type-safe orchestration of Python's ML/data science stack
- .NET: You want PureScript in enterprise/.NET environments (Excel, Azure, Unity)

The compilation strategy and performance characteristics are secondary to this ecosystem access.

13. Maturity & Maintenance

Backend	First Release	Current Status	Active Maintainers	Production Usage	Documentation
JS	2013	Stable	PureScript core team	Widespread	Comprehensive
Purerl	~2017	Stable	id3as team	Production (id3as)	Cookbook + README
Purescm	~2021	Experimental	Nathan Faubion	Internal (Arista)	Racket port report
Purepy	2025	Working	afc (solo)	Demo stage	ROADMAP + notes
.NET	—	Researched	—	—	Feasibility study
Lua	~2022	Experimental	Community	Testing	README + CLAUDE.md

Sustainability Factors

The backends that have survived are the ones with **production motivation**:

- Purerl: id3as uses it for real telecom/video systems
- Purescm/Purekt: Arista uses it internally

Backends without production users tend to stall when the maintainer's interest wanes or the PureScript compiler changes break compatibility.

14. Testing Infrastructure

Backend	Test Approach	Cross-Backend Comparison	Coverage
JS	Compiler test suite (<code>purs compile --run</code>)	Reference (defines correct behavior)	Full
Purerl	Forked package test suites + production use	Informal (against JS)	High
Purescm	Racket port test suite	Against JS reference (per report methodology)	Moderate

Backend	Test Approach	Cross-Backend Comparison	Coverage
Purepy	<code>test_ffi.py</code> (90 tests) + PureScript test modules	New: <code>cross_backend_test.py</code> (this session)	Moderate
.NET	—	—	—

The Case for Cross-Backend Testing

The Racket port report demonstrated that running the **same PureScript code** on both JS and the alternative backend, then diffing stdout, catches semantic divergences that unit tests miss. This is especially important for:

- String operations (encoding differences)
- Number formatting (float representation)
- Sort stability
- Error message formats

Purepy now adopts this approach with `cross_backend_test.py`, which runs each test module via both `node` and `python3` and produces a structured diff report.

15. Cross-Cutting Lessons for New Backends

These are the synthesized insights from studying all existing backends:

15.1 FFI Rewrite Cost is the Backend Killer

The FFI rewrite — not the code generator — is the largest single effort. Plan for 50+ FFI files covering prelude, effects, arrays, strings, ST, refs, exceptions, and more. Use the dependency-wave approach: implement in topological order, test each wave before starting the next.

15.2 Start from the Optimizer, Not Raw CoreFn

`purescript-backend-optimizer` provides aggressive inlining, uncurrying, dead code elimination, and pattern match optimization. Starting from optimized CoreFn rather than raw CoreFn means your code generator can be simpler, and generated code is faster from day one.

15.3 String Semantics: Investigate on Day One

If your target language's native string type isn't UTF-16, you will have divergences in `Data.String.CodeUnits`. Decide early: faithful UTF-16 reimplementaion (correct but complex) or accept divergence (simple but wrong for non-BMP characters). Document the decision prominently.

15.4 Cross-Backend Testing is Essential from the Start

Run the same PureScript code on both JS and your backend. Diff stdout. This catches semantic divergences that are invisible to unit tests, especially for:

- String operations with non-ASCII input
- Number formatting edge cases
- Sort stability assumptions
- Effect sequencing

15.5 Currying Tax on Static-Typed Targets

On dynamic targets (Python, Lua, Erlang, Scheme), closures are lightweight. On static targets (.NET, JVM), each closure is a heap-allocated delegate object. The backend-optimizer's uncurrying pass is **essential** on static targets — without it, performance will be unacceptable.

15.6 Plan for a Platform-Native Async Monad

Don't port Aff. Build a native async monad that wraps the target platform's concurrency primitives:

- Erlang: OTP processes
- Python: `asyncio`
- .NET: `Task<T>`
- Scheme: threads/futures

Aff's JavaScript event loop semantics don't translate; every backend that's tried has abandoned the port.

15.7 Reserved Word Audit

Every target language has reserved words that collide with PureScript identifiers. Common collisions: `not` (HeytingAlgebra), `and/or`, `class`, `type`, `import`. Implement escaping (typically append `_`) in the code generator from the start.

16. Implications for a .NET Backend

Applying all lessons from existing backends to the .NET feasibility assessment:

Advantages

- **Strings are free:** .NET's `System.String` is UTF-16, matching JavaScript exactly. `Data.String.CodeUnits` works with zero effort — the single most common pain point for other backends is absent.
- **Native TCO:** The CLR's `.tail` IL prefix provides native tail call support, if emitting CIL directly. No trampoline needed for mutual recursion.
- **Strong tooling:** Roslyn, NuGet, Visual Studio — mature ecosystem infrastructure.
- **NativeAOT:** Ahead-of-time compilation to native code could make this the fastest PureScript backend.

Challenges

- **Currying is the main performance risk:** `Func<>` delegates are heap-allocated. The backend-optimizer's uncurrying pass is essential. Expect 3-5x overhead without it, potentially near-JS with aggressive uncurrying.
- **Records have no natural row-polymorphic representation:** .NET has no structural typing for records. Options: `Dictionary<string, object>` (dynamic, like other backends), generated anonymous types (limited), or `ExpandoObject` (slow).
- **ADT representation requires class hierarchies:** F# proves this works (abstract base + subclass per constructor), but it's more verbose than the tagged-tuple approach used by dynamic backends.
- **F# competition:** F# already serves most ".NET + FP" needs. The case for PureScript must be specific: purity guarantees, effect tracking, type classes, and cross-backend code sharing.

Recommended Approach

1. **Start from the optimizer** — use `purescript-backend-optimizer`
2. **Emit CIL** (not C#) — for native TCO and closure control
3. **Plan for `Task<T>`** — don't port Aff
4. **Budget 50+ FFI files** — the dominant effort
5. **Target specific niches** — Excel embedding, Unity, cross-backend codebases
6. **Prototype with C# first** — easier to debug, accept no mutual TCO initially

Effort Estimate

Phase	Effort	Deliverable
Prototype (C# emitter, Prelude + Effect)	4-6 weeks	Hello World, basic types
Core FFI (arrays, strings, refs, ST)	4-6 weeks	Real programs compile

Phase	Effort	Deliverable
Optimization (uncurrying, DCE)	2-4 weeks	Acceptable performance
Ecosystem (JSON, HTTP, testing)	4-8 weeks	Practical use
CIL emitter (for TCO, performance)	4-8 weeks	Production-grade
Total	18-32 weeks	Full backend

Appendix: Quick Reference

How to Choose a Backend

If you need...	Use...	Because...
Web frontends	JS	Native browser support
Distributed systems	Purerl	OTP supervision, hot reload
ML/data science	Purepy	NumPy, Pandas, PyTorch access
Enterprise/.NET	.NET (future)	Azure, Unity, Excel
DSLs / language tooling	Purescm	Scheme macro system
Embedded / games	Lua	Lightweight runtime

File Naming Conventions

Backend	Module Data.Maybe	FFI File
JS	<code>output/Data.Maybe/index.js</code>	<code>src/Data/Maybe.js</code>
Purerl	<code>data_maybe@ps.erl</code>	<code>src/Data/Maybe.erl</code> (→ <code>data_maybe@foreign</code>)
Purepy	<code>data_maybe.py</code>	<code>data_maybe_foreign.py</code>
Lua	<code>Data.Maybe.lua</code>	<code>Data.Maybe.foreign.lua</code>